
Advanced Programming

ÁRPÁD TAMÁSI · ELTE FACULTY OF SOCIAL SCIENCES

SESSION 3 — 22 SEPTEMBER 2026

Where we left off

Vibe coding vs agentic engineering

VIBE CODING

- You look at **what it does**
- The prompt is the whole plan
- An error message goes straight back in
- Done when it seems to work

AGENTIC ENGINEERING

- You also look at **how it does it**
- **A spec**, then a plan, then small steps
- A failing test says what is wrong
- Done when **you can show it works**

Today’s session

	MIN	WHAT
A	30	Agentic engineering
B	45	Your turn — your own project
C	15	Homework

What is agentic engineering

Your brief is not a specification

You have all written a project brief. It says what the program is for.

It does not say **what the program does** — not precisely enough for someone else to build it.

The agent will fill every gap. It will not tell you that it did.

We call that filling **hallucination** when we disagree with it.

When we agree, we call it **intelligence**.

Three things a brief cannot say

YOUR BRIEF SAYS	IT CAN MEAN	OR IT CAN MEAN
the same reported time	the plane's own clock	the snapshot's clock
the CTR by industry	sum first → 2.00%	average the accounts → 4.67%
the example gives two cards	two, as written	five, by the rules above it

...and a third reading, and a fourth. **The agent picks one, and moves on.**

Fill the gaps before the code does

We fill them by writing **requirements**. Find every gap, and fill each one in this shape:

Requirement: CTR by industry

The dashboard SHALL compute an industry's CTR from the summed clicks and impressions of its accounts.

Scenario: Accounts of very different size

- **WHEN** they show 8000/400000, 2000/100000 and 10/100
- **THEN** the CTR is `2.00%`, not the average `4.67%`

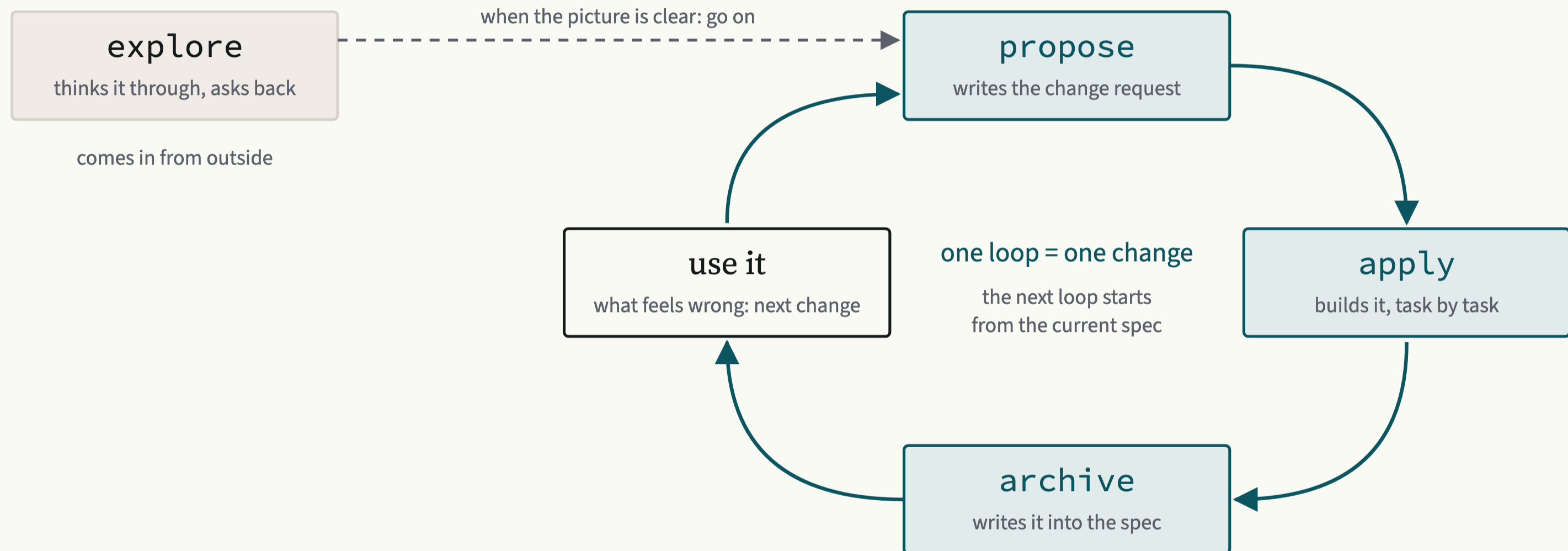
OpenSpec

A tool that asks

- It helps you find the gaps, by asking
- It will not ask everything — but its questions remind you of more. Say them
- It records your answers in the requirement format
- **This takes a while. See it through.**



The commands



1. explore — a conversation

It reads your README and your code, and asks. It never writes code.

The quick answer is tempting. You can also:

- ask back — why does it matter, what would you pick?
- ask for more, or talk it through when you are not sure
- ask for a screen or a chart — anything you can look at
- ask it to look at your data — and which method fits it

When the picture is complete, it offers to write the change request.

It asks, you answer

```
> /opsx:explore My Snake forgets the best score when  
I reload the page. What should we do about it?
```

```
Short version: your Snake doesn't forget the best score --  
it never had one.
```

```
Should the best score stay on this computer, or follow you  
to other devices (your phone, another laptop)?
```

```
> Just this computer. I only play it on my laptop.
```

It read the code first. Then it asked what only you could know.

Ask back

> Why does that matter? What would you pick, and why?

save at game over --> no game over happened
Best stays 12, the 14 is lost
save when beaten --> saved at 13, then at 14
after reload: Best 14

I'd pick: save the moment the score passes the best.

You get its reasons — and it tells you which part is your call.

Ask for more

```
> What do other Snake games do about this?
```

```
this is from general knowledge, not checked
```

```
...
```

```
| browser clones      | one best score      | the browser      |  
| arcade games       | top 10 + initials   | the machine      |
```

```
...
```

```
A top-10 table with initials exists because several  
people share one machine. Is that you?
```

It said what it did not check — and raised a question you had not asked.

Talk it through

> I'm not sure. Can we think it through – what happens if two people play on the same computer?

Anna plays score 20 --> Best: 20

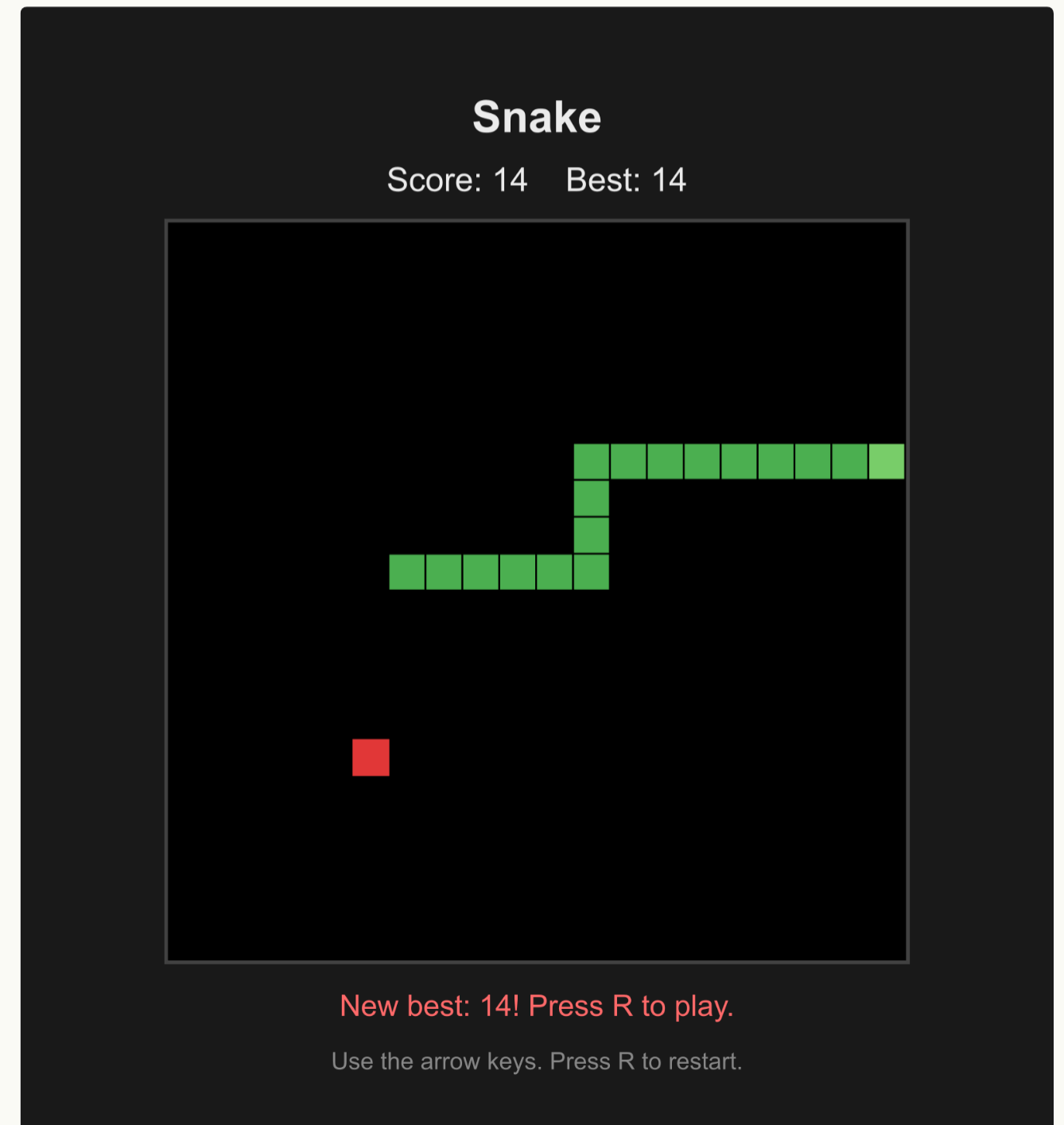
```
Ben plays      score 7  --> screen says Best: 20
                                   (that's Anna's number)
```

The browser remembers the computer, not the player.

A house record and a personal best are two different games.

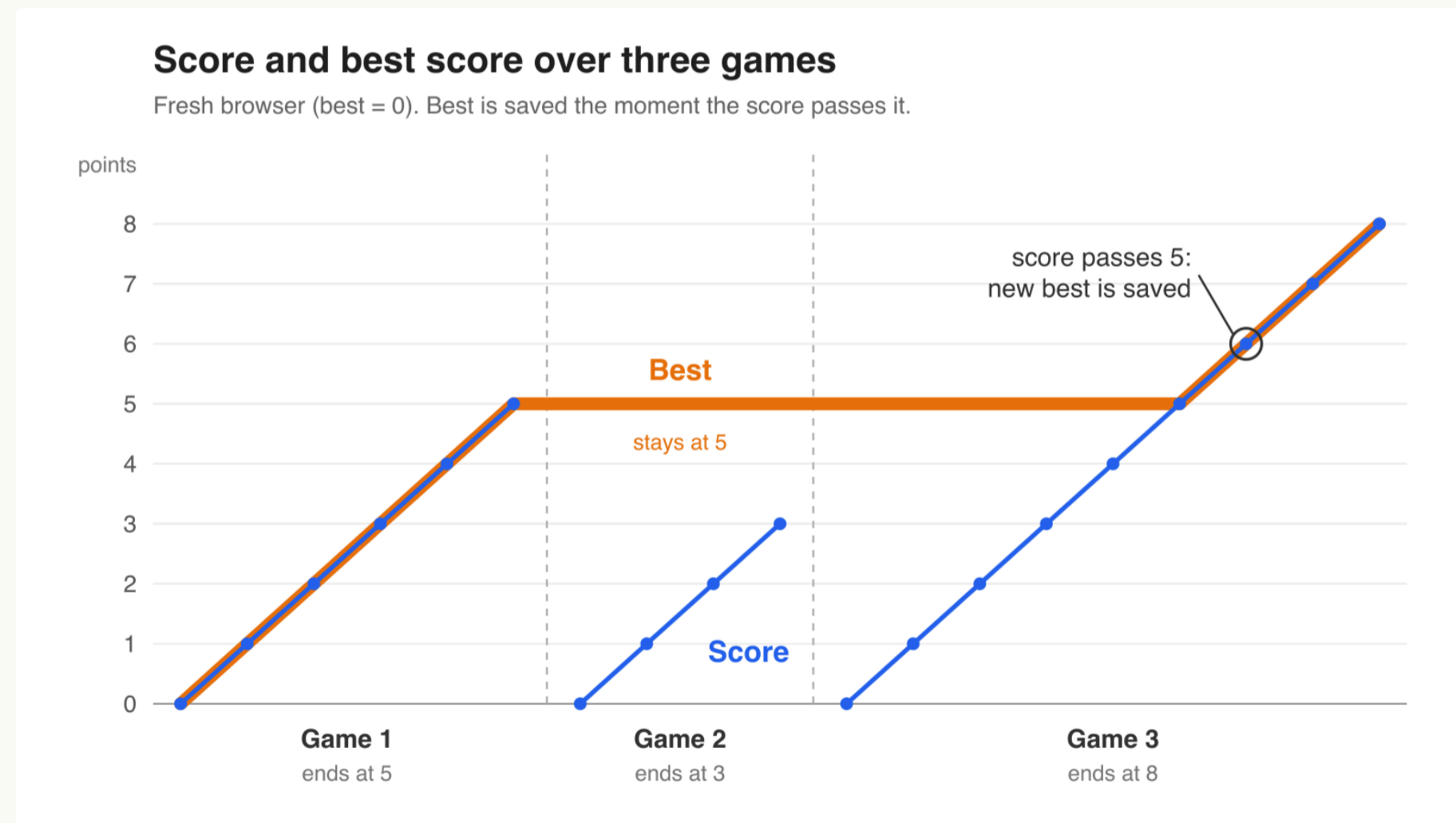
Ask for a screen

- *Can you show me the screen as a real picture, not text?*
- It drew it from the game's own code — and asked: “*your message line is red, the colour you use for ‘Game over!’*. A new record in red reads like bad news.”
- **A picture raises questions a sentence hides.**



Ask for a chart

And the chart as a real chart?



It noticed: *“tying your record is not a new best ... should a tie count too?”*

Ask it to look at your data

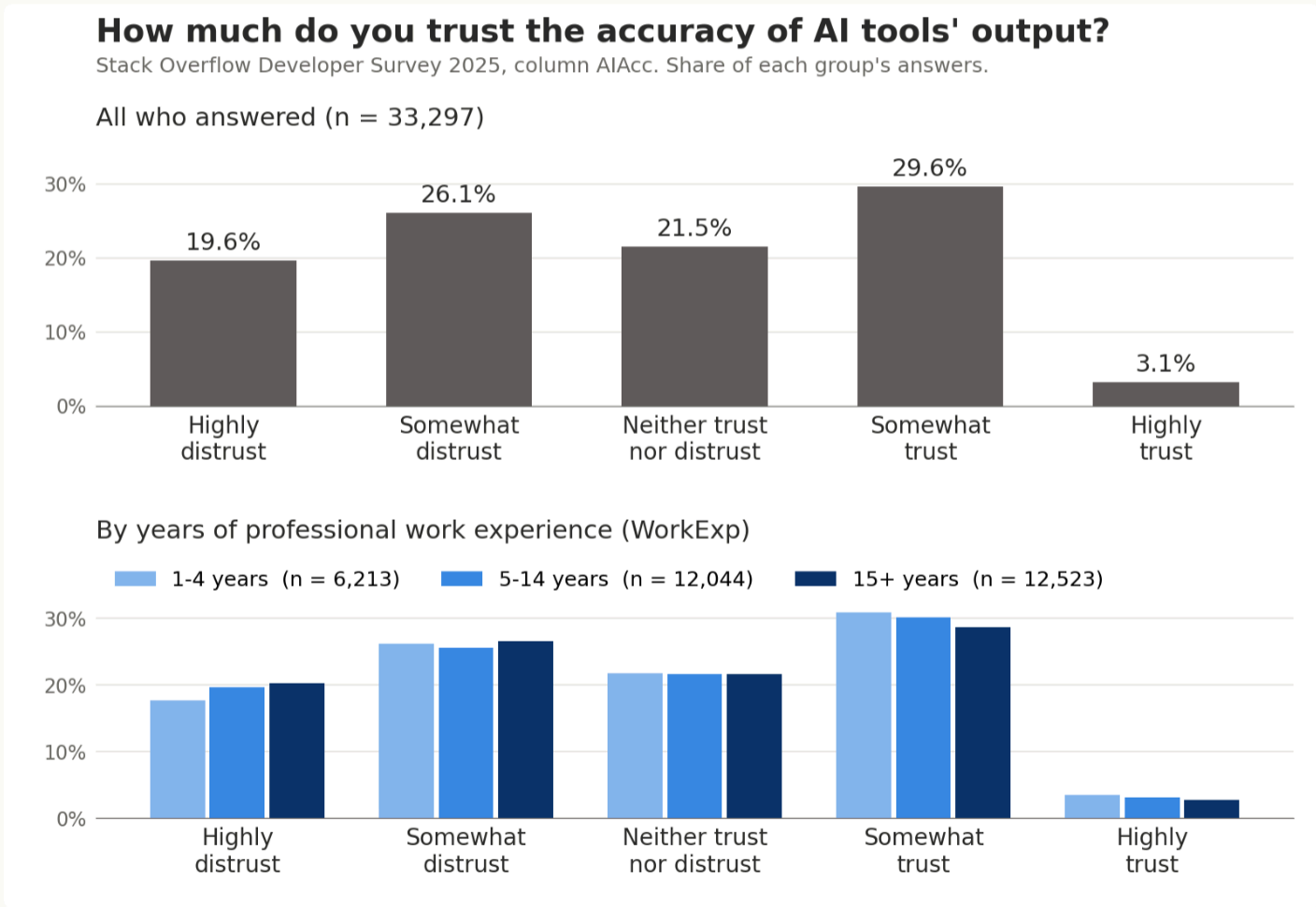
> Before we decide anything – look at the data.
What is actually in it?

Highly distrust	6,533	19.6
Somewhat distrust	8,685	26.1
Neither trust nor distrust	7,162	21.5
Somewhat trust	9,869	29.6
Highly trust	1,048	3.1
answered	33,297	(67.7% of all)

A third did not answer — and most of them left the survey, not this question.

Ask for the distribution

Show me the distribution of the trust answers, as a real chart.



“It is not a bell. There are two humps.” **Two camps — a mean would call it neutral.**

Ask which method fits

> Which method should I use to compare experience groups – and why that one?

1. The outcome is ordinal: five ordered labels, no distances. "Somewhat" to "Highly" is not a unit.

...

Kruskal-Wallis $H = 25.8$ $p = 2.5e-6$
 epsilon-squared = 0.0008

...

x t-test or ANOVA on the 1-5 codes – it treats the labels as equal steps

Every p-value is tiny and every effect is close to zero.

And more you can ask

ASK	FOR PROJECTS LIKE
<i>Look up the official rule. Where did you find it?</i>	sports statistics, tariffs, bank metrics
<i>Recompute the example in my README. Does it add up?</i>	word cards, a football predictor
<i>Check the API: what does it give, its limits, its cost?</i>	flight data, GitHub search, weather
<i>Make a fake sample with the same columns.</i>	bank exports, ad accounts, health data
<i>How many AI calls does one run make, and the cost?</i>	AI characters, AI personas
<i>What here counts as personal data?</i>	health, bank, CVs, photos
<i>Draw me who does what, in which order.</i>	multi-agent games, workflows, pipelines

2. propose — a change request

At the end of the conversation, explore offers: *“Do you want me to do that? Yes or no.”*
You say *go on*. It writes the change request — and one of its requirements:

```
### Requirement: Best score survives reload
The game SHALL keep the best score across page reloads.

#### Scenario: Page is reloaded
- **WHEN** the player reloads the page after a round
- **THEN** the best score still shows the stored value
```

Planning only — no code yet. It checks its own format: the shape, not the truth.

3. apply — building the change

```
> /opsx:apply
```

One task at a time, each ticked off:

```
- [x] 1.2 Raise the displayed best score whenever the current score  
    passes it, and verify it changes mid-round without losing  
  
    score = score + 1;  
+    updateBestScore();
```

Every task says how to check it. If the plan turns out wrong, change the plan.

4. archive — writing it into the spec

Like an amendment to a law: once passed, it is written into the consolidated text, and the amendment is filed.

```
> /opsx:archive
```

```
Applying changes to openspec/specs/best-score/spec.md:
```

```
+ 2 added
```

```
Change 'add-best-score' archived as '2026-09-16-add-best-score'.
```

The next change request is written against that text.

Your turn

45 minutes, your own project

1. Tell your agent to keep a **planning log**
2. **Install OpenSpec** — ask the agent to do it
3. Start **explore** on your README — say you want an MVP

Goal by the end of the session: explore offers to write your change request.

Help each other — **bonus points** for it.

First, tell it to keep a log

At the top of `AGENTS.md`, in your own repository:

Planning log

Whenever we decide something about this project – a requirement, a number, a name, a tool – append one line to `PLANNING_LOG.md`: the date, what was decided, and whether I decided it or you did. Never rewrite an earlier line.

And `CLAUDE.md` points to it, with one line: `@AGENTS.md`

Install OpenSpec

Same as last week: ask the agent.

```
Install OpenSpec version 1.13.0 for me, and run  
openspec init in this repository. Install Node first  
if it is missing. Then tell me what you changed.
```

What init leaves behind

One setup command. The agent runs it for you:

```
openspec init
```

```
openspec/
```

```
  config.yaml  ← settings; leave it as it is
```

```
  specs/      ← the truth: what the program does
```

```
  changes/    ← work in progress
```

The homework

By Saturday 20:00

Finish what you started today:

1. **explore** — until it offers to write the change request
2. **propose** — let it write the change request for your MVP
3. **apply** — let the agent build your MVP
4. **Commit and push.** What is not pushed does not exist

Hand in: the repository. In it, **the spec, the planning log, and the MVP.**

Stuck? Discuss it on Canvas — **bonus points** for helping others.

Have fun

Next time we meet, you
have your MVP running.

Thank you

Backup

What a command really is

Each command is a **skill**: a Markdown file the agent reads, then follows.

```
---  
name: openspec-propose  
description: Propose a new change with all artifacts  
generated in one step. Use when the user wants to  
quickly describe what they want to build ...  
---  
**Planning boundary**: This workflow creates planning  
artifacts only. ... Do not edit project code.
```

The description decides when it is used. The rest is the method.